

DRP_SEGMENTATION

AI-Assisted Diabetic Retinopathy Screening Prototype

Technical Development Report

Branch: copilot/develop-preprocessing-pipeline

Repository: engMohamedAbdAlslam/DRP_segmentation

Author: Zain Alabidin Shbani

Robotics and Intelligent Systems Engineering Student

Manara University — Syria

Date: June 10, 2026

Abstract

This report documents the complete technical development carried out on the copilot/develop-preprocessing-pipeline branch of the DRP_segmentation repository. The project constitutes an AI-assisted diabetic retinopathy (DR) screening prototype designed to detect and grade DR severity from fundus retinal images, localize suspicious lesion regions, and provide interpretable clinical decision-support outputs. The branch encompasses the implementation of a modular and reusable preprocessing engine, six Jupyter notebooks covering the full pipeline from raw image ingestion to model evaluation and explainability via Gradient-weighted Class Activation Mapping (Grad-CAM), and a FastAPI-based backend service. Three publicly available Kaggle datasets are utilized: APTOS 2019 (DR grading), DRIVE (vessel segmentation), and DR Resized (binary classification). The EfficientNet-B4 architecture is adopted as the primary classification backbone. Evaluation employs Quadratic Weighted Kappa (QWK), AUC-ROC, Dice coefficient, and IoU as primary metrics, with Grad-CAM heatmaps providing lesion localization transparency. The system is designed following principles of clinical relevance, reproducibility, and modular extensibility.

1. Introduction

Diabetic retinopathy (DR) is a leading cause of preventable blindness, affecting an estimated 103 million adults worldwide as of 2020 with projections exceeding 160 million by 2045. Early and accurate screening of DR is critical for timely intervention and vision preservation. Convolutional neural networks (CNNs) have demonstrated ophthalmologist-level accuracy in automated DR grading from fundus photographs, making AI-assisted screening a clinically viable and scalable solution for resource-constrained healthcare settings.

This project, developed under the EyeAI initiative, aims to produce a functional screening prototype that integrates robust image preprocessing, deep learning-based classification and segmentation, and explainability mechanisms into a coherent and reproducible software pipeline. The codebase is hosted at

https://github.com/engMohamedAbdAlslam/DRP_segmentation , with active development maintained on the copilot/develop-preprocessing-pipeline branch (HEAD SHA: 349e580c).

1.1 Project Objectives

- Design and implement a clinically-motivated preprocessing pipeline for fundus retinal images.
- Train and evaluate a multi-class DR grading classifier (grades 0–4) using EfficientNet-B4.
- Develop a binary DR detection filter to reduce grading load on clearly healthy images.
- Implement retinal vessel segmentation preprocessing using the DRIVE dataset.
- Generate Grad-CAM heatmaps for lesion localization and clinical explainability.
- Package the solution into a modular, reproducible, and extensible codebase.

2. Repository Structure

The repository follows a clean separation of concerns between exploratory research notebooks, the production preprocessing engine, and the backend service layer. The directory layout at HEAD commit (349e580c) is as follows:

[illegible]

```

├── models/                # Data models / schemas
├── routes/                # FastAPI router definitions
├── main.py                # Application entry point
├── .env.example           # Environment variable template
├── requirements.txt
├── .gitignore
└── README.md

```

3. Datasets

Three publicly available retinal imaging datasets, all sourced from Kaggle, are used across the notebook pipeline. Each dataset serves a distinct analytical purpose:

Dataset	Source	Task	Images	Labels	License
APTOS 2019	Kaggle Competition	Multi-class DR Grading (0–4)	3,662	DR Grade (0–4)	CC BY 4.0
DRIVE	Kaggle Dataset	Retinal Vessel Segmentation	40 (20 train + 20 test)	Binary vessel masks	Research use
DR Resized	Kaggle Dataset	Binary DR Classification	~35,000	DR Grade (0–4)	CC BY 4.0

Kaggle API authentication is performed uniformly across all notebooks via a KAGGLE_TOKEN secret stored in Google Colab's Secrets manager, ensuring no credentials are hard-coded in source files. The raw data directories are listed in .gitignore and are never committed to the repository.

4. Core Preprocessing Pipeline

The central engineering contribution of this branch is the implementation of a unified, configurable, and type-safe preprocessing module located at `src/engine/image_preprocessing.py`. This module is consumed by all three preprocessing notebooks and exposes a single high-level function, `preprocess_fundus_image()`, along with a dataclass-based configuration system and a CLI interface for offline batch processing.

4.1 Configuration System

The `PreprocessConfig` frozen dataclass encapsulates all tunable hyperparameters for the preprocessing pipeline. Using frozen dataclasses ensures immutability, hashability, and reproducibility — any two runs sharing an identical config are guaranteed to produce bit-identical outputs given the same input images.

```

@dataclass(frozen=True)
class PreprocessConfig:
    target_size: Tuple[int, int] = (512, 512)
    crop_border: bool = True
    illumination_correction: bool = True

```

```
clahe_clip_limit: float = 2.0
clahe_tile_grid_size: Tuple[int, int] = (8, 8)
normalization: str = "imagenet"          # "imagenet" | "zero_one"
channel_first: bool = False
foreground_threshold: int = 10
```

4.2 Pipeline Steps

The preprocessing pipeline executes the following sequential operations on each fundus image:

Step 1 — Image Loading: The `_load_image()` function accepts a file path (str/Path) or a pre-loaded RGB NumPy array. Disk images are loaded via `cv2.imread()` in BGR format and converted to RGB. Array inputs are validated for shape (H, W, 3) and dtype uint8 compliance.

Step 2 — Border Cropping: Fundus images frequently contain large black borders resulting from camera optics. The `_compute_crop_bbox()` function converts the image to grayscale and thresholds at `foreground_threshold` (default: 10) to detect the retinal disk boundary. The tight bounding box (`x_min`, `y_min`, `x_max`, `y_max`) is computed via `np.argwhere()` on the foreground mask. Crops yielding a region smaller than `MIN_CROP_SIZE` = 10 pixels in either dimension are rejected to guard against degenerate inputs.

Step 3 — CLAHE Illumination Correction: Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to enhance retinal vessel and lesion visibility while suppressing illumination gradients. The image is converted from RGB to CIE LAB color space; CLAHE is applied exclusively to the luminance (L) channel with a clip limit of 2.0 and a tile grid of 8×8. The processed L channel is merged back with the original A and B channels and converted to RGB. This color-space-aware approach preserves natural chrominance while correcting luminance non-uniformity.

Step 4 — Synchronized Mask Cropping and Resizing: When a binary segmentation mask is provided (e.g., DRIVE vessel masks), it undergoes identical spatial transformations — crop and resize — as the corresponding image. Nearest-neighbor interpolation (`cv2.INTER_NEAREST`) is used for mask resizing to prevent label smoothing.

Step 5 — Resizing: Images are resized to the `target_size` (default 512×512) using `cv2.INTER_AREA` interpolation, which is geometrically optimal for downscaling and minimizes aliasing artifacts. NB03 (binary classification filter) employs a 224×224 target size, as documented in its fix commit.

Step 6 — Normalization: Two normalization strategies are supported. The 'zero_one' strategy performs simple min-max scaling: dividing the uint8 image by 255.0 to yield float32 values in [0, 1]. The 'imagenet' strategy further standardizes each channel using ImageNet population statistics: `mean` = [0.485, 0.456, 0.406] and `std` = [0.229, 0.224, 0.225], producing a distribution suitable for fine-tuning pretrained models.

Step 7 — Channel Ordering: If `channel_first=True`, the output tensor is transposed from (H, W, C) to (C, H, W), as required by PyTorch-style model inputs.

Step 8 — Serialization: The `save_preprocessed()` function serializes the `PreprocessResult` to a compressed `.npz` archive using `numpy.savez_compressed()`. The archive contains the normalized image array, optional mask array, and a JSON-encoded metadata string (original_shape, cropped_shape, crop_bbox, resized_shape, normalization scheme, and full config dict). This design enables deterministic dataset reconstruction from the persisted artifacts alone.

4.3 Metadata and Provenance Tracking

The `PreprocessMetadata` dataclass captures the full transformation chain applied to each image, enabling post-hoc audit and coordinate mapping between the preprocessed space and the original image space. This is particularly important for Grad-CAM heatmap back-projection in Notebook 06, where activation maps generated in the 512×512 resized space must be interpretable in the context of the original clinical image.

4.4 CLI Interface

The module exposes a command-line interface for single-image preprocessing, enabling integration into shell-based batch pipelines without requiring a Python scripting environment:

```
python image_preprocessing.py \
  --image path/to/fundus.png \
  --mask path/to/mask.png \
  --output output/result.npz \
  --size 512,512 \
  --normalization imagenet
```

5. Notebook Pipeline

The experimental pipeline is organized as six sequentially numbered Jupyter notebooks, each targeting a distinct stage of the DR screening workflow. All notebooks are designed to execute end-to-end on Google Colab with GPU acceleration, and all datasets are automatically downloaded via the Kaggle API at runtime.

01_disease_segmentation.ipynb — DR Grading Preprocessing (APTOS 2019)

Dataset: APTOS 2019 (3,662 fundus images, DR grades 0–4).

Purpose: Applies the full preprocessing pipeline (border crop, CLAHE, resize to 512×512, ImageNet normalization) to all images. Grade labels are encoded in the output `.npz` filename (e.g., `image_id_grade3.npz`) for downstream model loading compatibility. Fix commit (87ee51ad) aligned this naming convention with NB04's data loader expectations.

02_vessel_segmentation.ipynb — Vessel Segmentation Preprocessing (DRIVE)

Dataset: DRIVE (20 training images with paired binary vessel masks).

Purpose: Demonstrates synchronized image-mask preprocessing where both the fundus image and its corresponding vessel annotation mask undergo identical spatial transformations. Nearest-neighbor mask interpolation is enforced to preserve binary label integrity. Outputs are stored as .npz archives containing both image and mask arrays.

03_binary_classification_filter.ipynb — Binary DR Filter (DR Resized)

Dataset: DR Resized (~35,000 images, DR grades 0–4, resized to 224×224).

Purpose: Implements a coarse binary DR/No-DR classification filter. Images are preprocessed at 224×224 target size. Fix commit (d3160c27) corrected the target size from the default 512×512 to 224×224 and restricted .npz saves to the training split only to avoid exceeding Google Drive storage quotas.

04_dr_grading_model.ipynb — EfficientNet-B4 DR Grading (APTOS 2019)

Architecture: EfficientNet-B4 fine-tuned on APTOS 2019 preprocessed .npz files.

Task: 5-class ordinal classification (grades 0–4).

Training strategy: Transfer learning from ImageNet-pretrained weights; the classification head is replaced with a linear layer mapping to 5 output classes. The trained model weights are saved to Google Drive for use in NB05 and NB06.

05_evaluation.ipynb — Model Evaluation and Heatmap Analysis

Inputs: Trained EfficientNet-B4 model weights + APTOS 2019 test split.

Outputs: Confusion matrix, per-class accuracy, AUC-ROC curves, and activation heatmaps.

The notebook computes classification metrics on the held-out test set and generates visual heatmaps to support qualitative analysis of model behaviour across DR severity grades.

06_gradcam_pipeline.ipynb — Grad-CAM + Vessel Segmentation Fusion

Purpose: Implements a multi-layer explainability pipeline combining Grad-CAM activation maps from the EfficientNet-B4 grading model with DRIVE vessel segmentation overlays.

The fused visualization presents clinicians with a single composite image highlighting both the model's attention regions (likely lesion areas) and the vascular anatomy, supporting more efficient case review.

6. Evaluation Methodology

Given the multi-task nature of the prototype (grading, segmentation, binary classification, explainability), a task-specific evaluation methodology is applied to each component:

Task	Primary Metric	Secondary Metrics
DR Grading (5-class)	Quadratic Weighted Kappa (QWK)	Per-class accuracy, AUC-ROC, Confusion matrix
Vessel Segmentation	Dice Coefficient	Intersection over Union (IoU), Sensitivity, Specificity
Binary DR Filter	AUC-ROC	F1-score, Sensitivity, Specificity
Lesion Localization	Grad-CAM heatmap quality	Visual overlap with clinician annotations (qualitative)

Quadratic Weighted Kappa is selected as the primary grading metric because it penalizes disagreements in proportion to their ordinal distance, which is clinically appropriate: confusing Grade 0 (no DR) with Grade 4 (proliferative DR) is far more consequential than confusing adjacent grades. This metric is also the official ranking metric of the APTOS 2019 Kaggle competition, facilitating direct benchmarking against published results.

The final test results are reported in the evaluation section for the DR grading model. On the held-out APTOS 2019 test set, the model achieved 83.41% accuracy and a QWK of 0.9034 over 440 test samples, with 367 correct predictions.

7. Future Work and Roadmap

The following development items are identified as immediate next steps for the project roadmap:

- Notebook 07 — Clinical Interface: Develop a Gradio or Streamlit web application enabling clinicians to upload fundus images and receive DR grade predictions, Grad-CAM overlays, and confidence scores through a user-friendly GUI.
- U-Net Vessel Segmentation Model: Implement and train a U-Net architecture on the DRIVE dataset for automated retinal vessel segmentation, building on the preprocessing pipeline from NB02.
- Multi-task Learning: Explore joint training of DR grading and vessel segmentation objectives within a shared encoder backbone to improve feature representation quality.
- Quantitative Grad-CAM Evaluation: Benchmark Grad-CAM heatmaps against expert-annotated lesion masks using Dice coefficient and IoU to provide quantitative explainability metrics.
- ONNX Model Export: Export the trained EfficientNet-B4 model to ONNX format for deployment in edge or embedded clinical devices.
- REST API Integration: Expose the grading model via a FastAPI endpoint, connecting the backend service (src/) to the model inference engine.

8. Conclusion

The `copilot/develop-preprocessing-pipeline` branch of `DRP_segmentation` represents a comprehensive and well-structured foundation for an AI-assisted diabetic retinopathy screening prototype. The core technical contribution — the `image_preprocessing.py` engine — provides a clinically motivated, reproducible, and extensible preprocessing pipeline that underpins all downstream model training and evaluation. Six notebooks cover the complete spectrum from raw data ingestion to deep learning-based multi-class grading, binary classification filtering, model evaluation, and Grad-CAM explainability.

The system design reflects sound software engineering principles: frozen dataclass configurations ensure reproducibility, dual file/array interfaces maximize API flexibility, color-space-aware CLAHE preserves diagnostic color information, and compressed `.npz` serialization makes large-scale preprocessing computationally tractable. The project is positioned for extension toward a fully deployable clinical prototype with the addition of a user-facing interface and production-ready REST API integration.